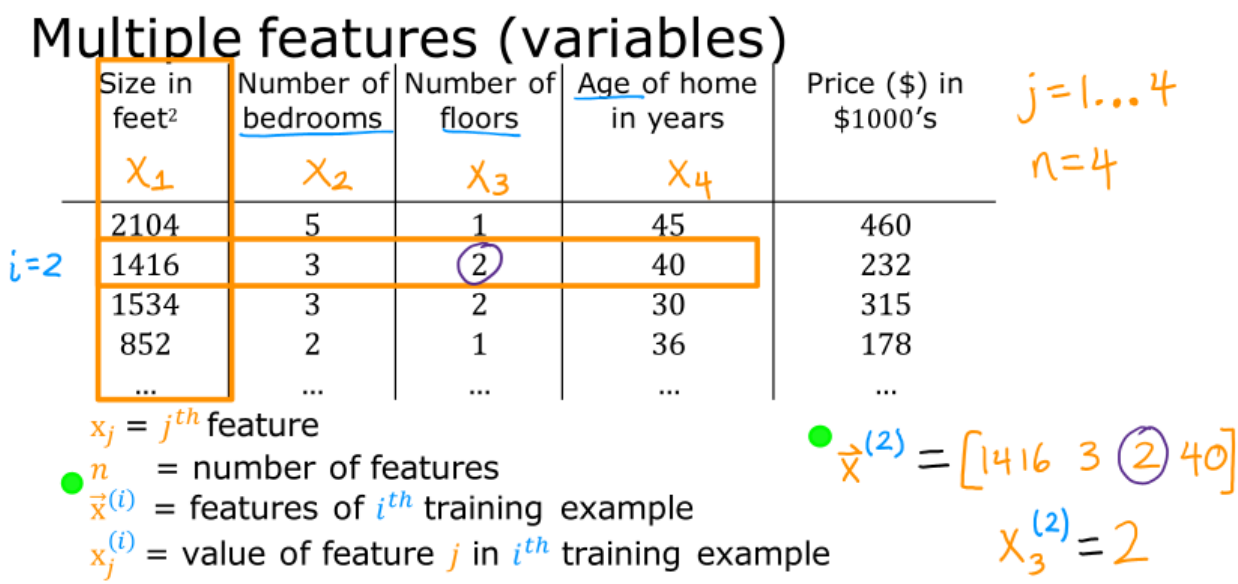
C1_Week2

Linear Regression With Multiple Variables

Trong tuần một mình đã học về Linear Regression với một ẩn, một cặp tham số (w,b). Những trong thực tế một Model Machine Learning sẽ đòi hỏi nhiều chi tiết hơn từ đầu vào, chẳng hạn như khi mua nhà, mình cần cân nhắc kích thước, số tầng, số phòng ngủ, ... của chiếc nhà nên tuần này sẽ học về Linear Regression with Multiple Variables.



Ví dụ về các input của một mô đình dự đoán giá nhà, cùng với cấc ký hiệu cần nhớ.

Nhắc lại Model Linear Regression 1 ẩn tuần trước:

$$f_{w,b}(x) = wx + b$$

Model Linear Regression nhiều ẩn sẽ như sau:

$$f_{w,b}(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Mình có thể viết dưới dạng tổng sigma:

$$f_{w,b}(\mathbf{x}) = \sum_{i=1}^{n} w_i x_i + b$$

Sau này sẽ sử dụng sự biểu diễn dưới dạng vector ngắn gọn nhiều hơn:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Vectorization

Vector hoá nói về việc sử dụng Vector để tăng tốc độ học máy, tối ưu hoá việc viết code cũng như là trong quá trình tính toán.

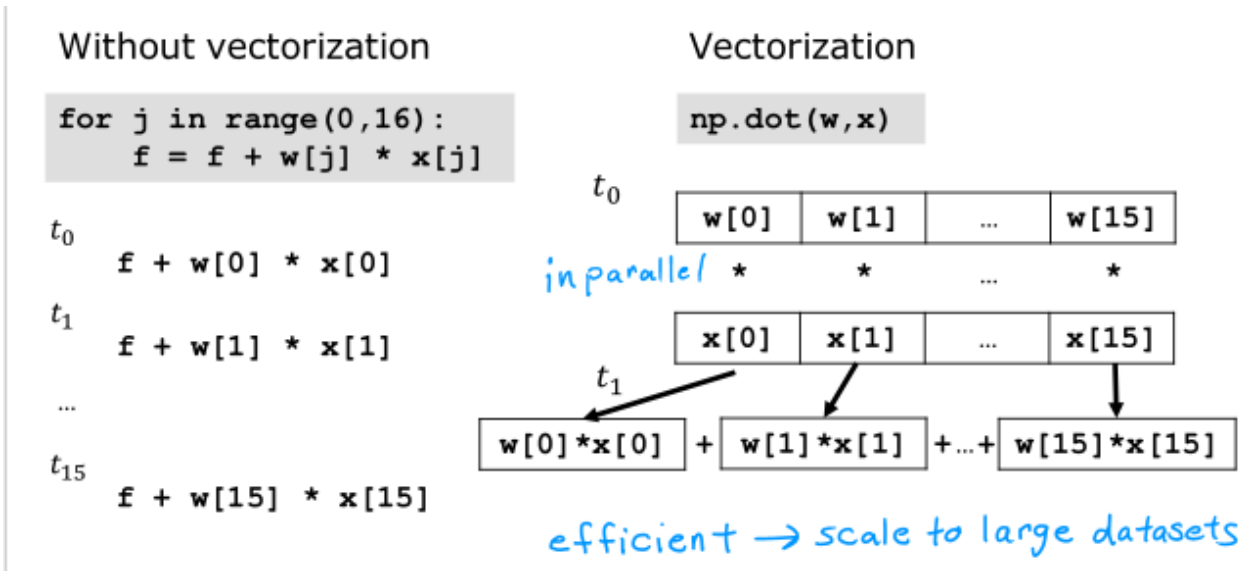
- Code theo vòng lặp: (chạy theo từng vòng lặp chậm, không phù hợp khi có nhiều training data)

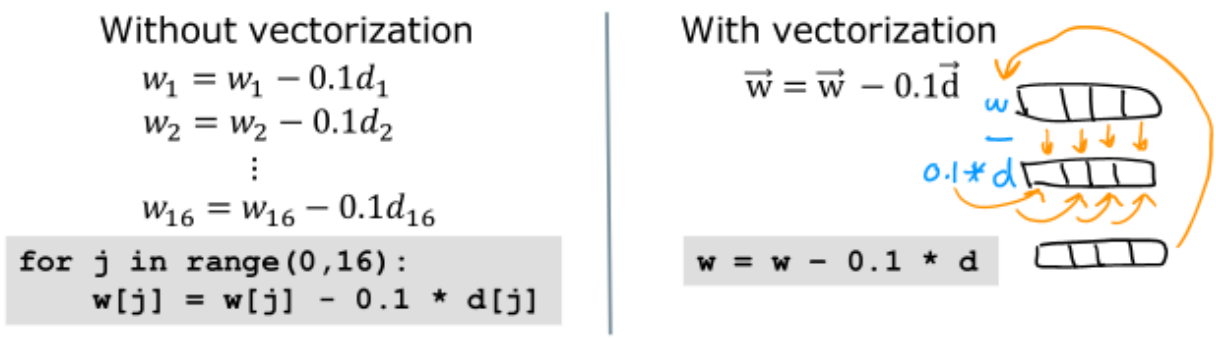
```
f = 0
for j in range(0, 3):
    f = f + w[j] * x[j]
f = f + b
```

- Code sử dụng vector hoá: (ngắn và dễ hiểu hơn)

```
import numpy as np
w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])
f = np.dot(w, x) + b
```

→ Một lý do quan trọng trên hết mà buộc mình phải sử dụng vector hoá là việc sử dụng np.dot(w,x) giúp tận dụng tốt hơn phần cứng của máy tính, tạo ra một khác biệt lớn khi có một training set lớn trên thực tế.





Gradient Descent for Multiple Regression

Đây là phần ứng dụng GD cho Multiple Regression.

- Nhắc lại Model của mình:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

- Gradient Descent theo vector: (giống với 1 ẩn, giờ thêm vector)

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

- Mình sẽ cập nhật với công thức:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

→ Giống như Gradient Descent với 1 feature, mình vẫn phải CẬP NHẬT ĐỒNG THỜI (Simultaneous Update).

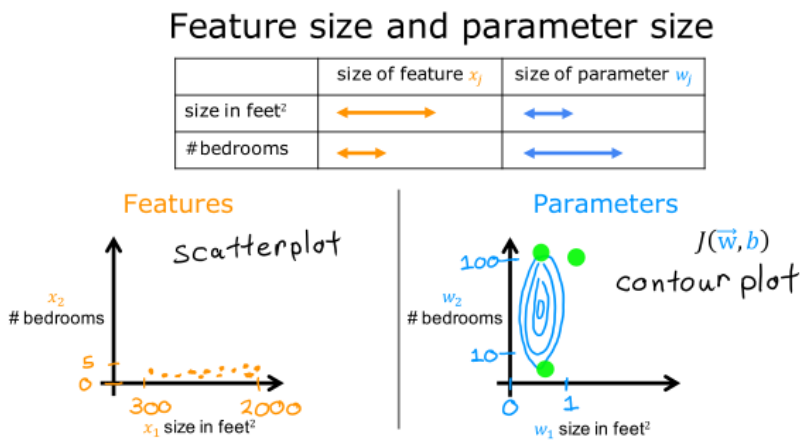
Note:

- Ngoài Gradient Descent, mình cũng có một số cách khác để train một mô hình prediction, một trong số đó là Normal Equation.
- Một số lưu ý với Normal Equation:
  - Normal Equation chỉ có thể được sử dụng cho mô hình Linear Regression
  - Normal Equation có thể tìm ra được w và b tối ưu mà không cần sử dụng vòng lặp.
  - Normal Equation sẽ chạy chậm khi có nhiều feature (nhiếu biến input) khác nhau (>10000).

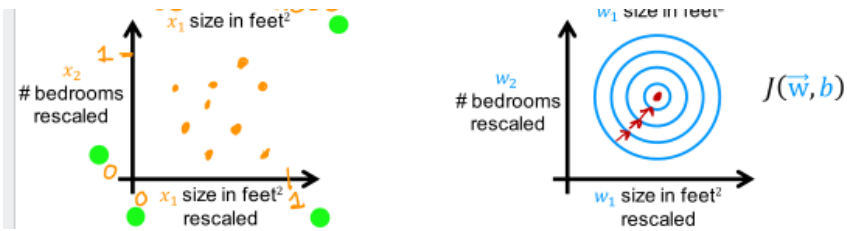
Feature Scaling

Feature Scaling nói về việc mình phóng to hoặc thu nhỏ một feature về một khoảng nhất định (thường là nhỏ) để Model của mình dễ sử dụng hơn.

- Ví dụ như khi mình có một feature “kích thước căn nhà” với khoảng giá trị từ 300 - 2000, feature “số phòng ngủ” với khoảng giá trị từ 0 - 5, hậu quả là khi phác hoạ Cost Function sẽ bị kéo dài ra (kiểu cái bát biến dạng), khi chạy Gradient Descent sẽ chậm hơn.



→ Để khắc phục, mình sẽ cần chuẩn hoá (Standardization) các feature của mình sao cho khi vẽ lại sẽ được như này:



1. Chia cho giá trị lớn nhất:

- Chia cho giá trị lớn nhất giúp các feature khi đã rescale luôn bé hơn 1.

$$300 \le x_1 \le \underline{2000} \qquad 0 \le x_2 \le \underline{5}$$

$$x_{1,scaled} = \frac{x_1}{\underset{max}{2000}} \qquad x_{2,scaled} = \frac{x_2}{\underset{max}{5}}$$

$$0.15 \le x_{1,scaled} \le 1 \qquad 0 \le x_{2,scaled} \le 1$$

2. Mean Normalization

- Lấy hiệu của input với giá trị trung bình chia cho hiệu max và min, đưa các giá trị thường về khoảng [-1, 1] (chắc thế)

$$300 \le x_1 \le 2000 \qquad 0 \le x_2 \le 5$$

$$x_1 = \frac{x_1 - \mu_1}{\underbrace{2000-300}_{\text{max-min}}} \qquad x_2 = \frac{x_2 - \mu_2}{\underbrace{5-0}_{\text{max-min}}}$$

$$-0.18 \le x_1 \le 0.82 \qquad -0.46 \le x_2 \le 0.54$$

3. Z-score Normalization (dùng nhiều)

https://www.geeksforgeeks.org/data-analysis/z-score-normalization-definition-and-examples/

$$300 \le x_1 \le 2000 \qquad 0 \le x_2 \le 5$$

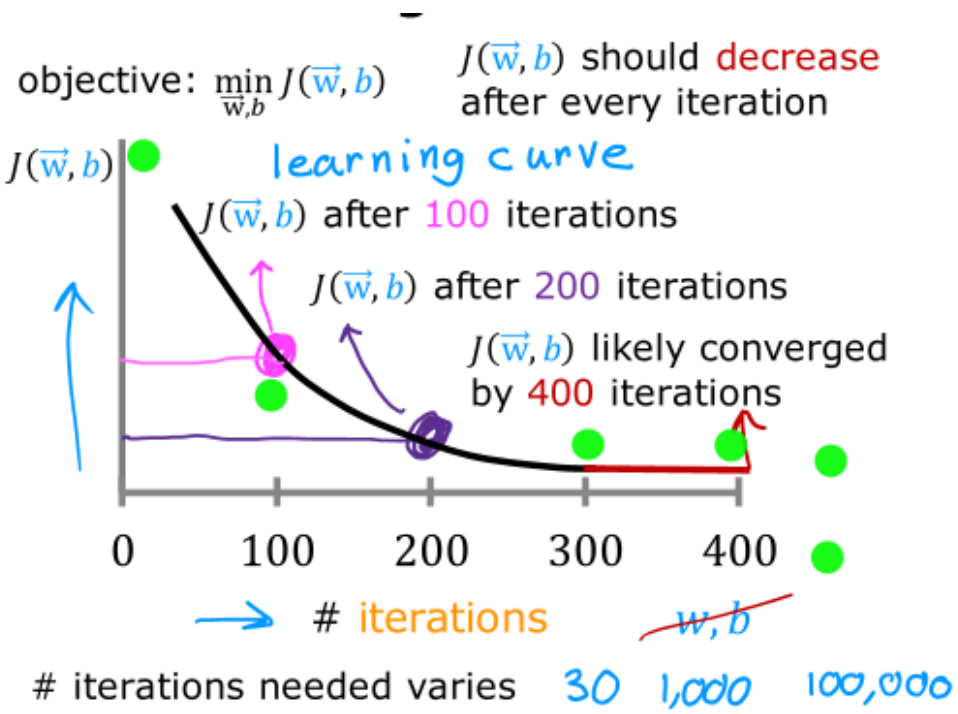
$$x_1 = \frac{x_1 - \mu_1}{\sigma_1} \qquad x_2 = \frac{x_2 - \mu_2}{\sigma_2}$$

$$-0.67 \le x_1 \le 3.1 \qquad -1.6 \le x_2 \le 1.9$$

→ Mục tiêu của Feature Scaling là mình đưa các giá trị input vào các khoảng nhỏ chấp nhận được, chẳng hạn [-1,1], [-3,3], [-0.3, 0.3], [0, 3], [-2, 0.5],...

Checking Gradient Descent for Convergence

Phần này là để kiểm tra xem liệu Gradient Descent của mình đang hoạt động đúng hay không. Do Gradient Descent được dùng để giảm Cost Function nên để kiểm tra, mình nên kiểm tra xem Cost có giảm qua các lần lặp hay không.



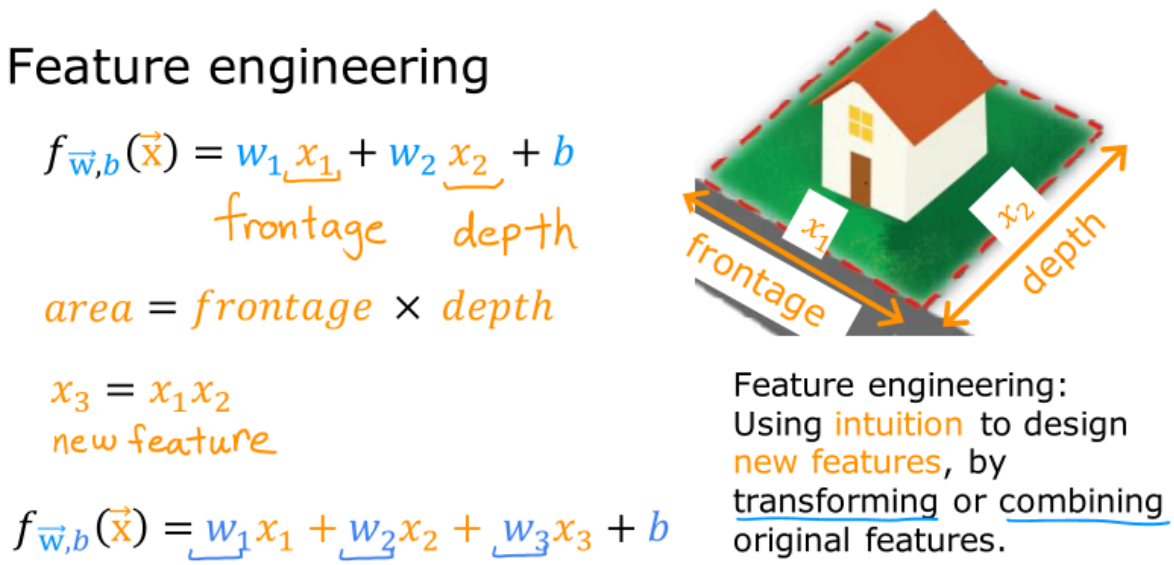
Khi mình đã biết là Cost đang giảm, mình cần phải biết khi nào Gradient Descent đã converge. Mình sẽ so sánh sự giảm của Cost qua mỗi lần lặp, nếu nhỏ hơn một giá trị **ε =** 10^-3 thì khẳng định là đã converge. Cos

---

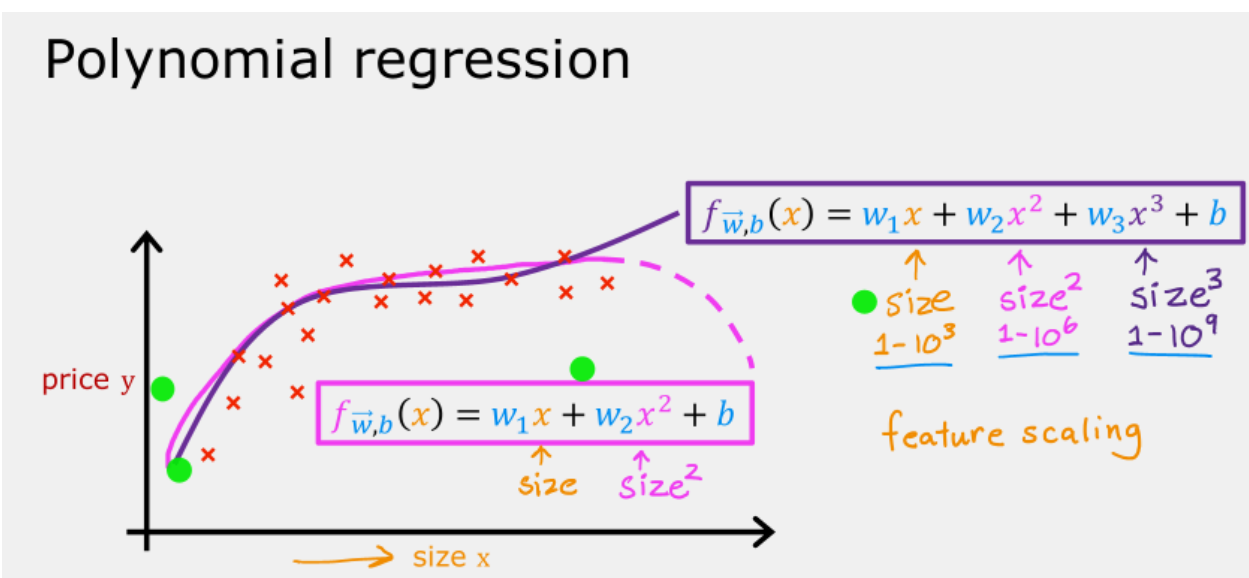
Feature Engineering

Phần này không dài, Feature Engineering nói về việc mình sử dụng trực giác để tạo ra các features mới, giúp ích trong việc học máy. Một số các cách mình có thể làm như vậy bằng cách biến đổi, hoặc cộng gộp các feature đã có sẵn.

Trong slide có một ví dụ:



→ Nhận thấy mình có thể tạo ra một feature mới là “diện tích nhà” bằng việc nhân 2 feature “mặt tiền” và “độ sâu” với nhau.

Polynomial Regression

Ngoài Linear Regression bậc nhất, mình cũng có một số Model khác là Polynomial Regression, trong đó có các dạng đa thức bậc cao hơn. Khi mình vẽ hình ra thì mô hình đa bậc có thể tạo ra các đường cong mà có thể phù hợp với Training Set hơn một đường thẳng.